

Linguaggio C - I file



File di testo e file binari



Due tipi di file:

file di testo trattati come sequenze di caratteri e organizzati in linee (ciascuna terminata da '\n')

file binari visti come sequenze di bit

Per gestirli bisogna includere la libreria **stdio.h**



I file di testo

Struttura sequenziale dei file



I file hanno una struttura sequenziale cioè:

i valori sono organizzati in una sequenza e per accedere ad un particolare valore è necessario "**scorrere**" tutti quelli che lo precedono.

Per accedere ad un file da un programma C, è necessario predisporre una **variabile cioè un puntatore al file**

Il puntatore al file



Viene definito prima dell'utilizzo del file dalla seguente istruzione:

FILE *fp

fp è il puntatore al file. Esso indica il file e l'elemento corrente all'interno del file.

Operazioni sui file



- **Apertura:** associazione del programma ad un file di riferimento
- **Lettura:** trasferimento di valori presenti nel file in una o più variabili del programma
- **Scrittura:** trasferimento del contenuto di una o più variabili del programma nel file
- **Chiusura:** termine delle operazioni sul file

Libreria standard in C – file testo



La libreria standard del linguaggio C rende disponibili le seguenti funzioni per la gestione dei file di testo

Operazione	Funzione file testo
Apertura	fopen
Lettura	fscanf
Scrittura	fprintf
Chiusura	fclose

fopen



```
fp=fopen("nome_file","modalità_util");
```

"nome_file" è il nome del file che si vuole aprire

"modalità_util" può essere:

- "**rt**" apertura del file in **lettura**
- "**wt**" apertura del file in **scrittura**
- "**at**" apertura del file in **append (aggiunta)**

fopen



- “**r**” apertura in lettura (read). Se il file non esiste la lettura fallisce.
- “**w**” apertura di un file vuoto in scrittura (write). Se il file esiste il suo contenuto viene cancellato.
- “**a**” apertura in aggiunta (append). Crea il file se non esiste.

Seguiti da:

“**t**” apertura in modalità testo (default) o “**b**” modalità binaria ed eventualmente da “**+**” apertura con possibilità di lettura e scrittura.

Esempi



`FILE *fp=fopen("input.txt","wt");` apertura di input.txt in scrittura

`FILE *fp=fopen("input.txt","rt");` apertura di input.txt in lettura

`FILE *fp=fopen("input.txt","at");` apertura di input.txt in append

IMPORTANTE



Se l'operazione di apertura va a buon fine ritorna come risultato un puntatore al file aperto.

Se l'apertura fallisce, fopen restituisce il valore NULL.

Motivi di fallimento nell'apertura del file:

- Il file non esiste (o non è nella cartella)
- Il file viene aperto in una modalità incompatibile con le sue proprietà (ad esempio: apertura in scrittura di un file read only)

Quindi verificare sempre che il file sia aperto correttamente (testando se il puntatore vale NULL)



```
#include <stdio.h>

int main()
{
    FILE *fp
    fp=fopen("input.txt","rt");    // apro il file input.txt in lettura
    if(fp==NULL)                  // controllo che l'apertura vada a buon fine
    {
        printf("Errore apertura file");
        return 1;
    }
    while((fscanf(fp,"%d",&num))>0) // finché la lettura avviene con successo
    {
        .....
    }
}
```

Lettura da un file aperto in lettura - fscanf



L'istruzione per leggere un file (aperto in lettura) è:

```
fscanf (fp,"%d",&num)
```

Dove:

fp puntatore al file

“%d” indica il codice formato di un intero

&num indica l'indirizzo della variabile che conterrà il valore letto dal file

Scrittura su un file aperto in scrittura - fprintf



L'istruzione per **scrivere** in un file è:

```
fprintf(fp,"%d",num);
```

Dove:

fp puntatore al file

"%d" indica il codice formato di un intero

num indica la variabile che contiene il valore da scrivere nel file

fclose



Alla fine delle operazioni è necessario chiudere il file:

fclose(fp);

Perchè è importante chiudere il file?

La chiusura del file permette, a qualcun altro, di poterlo riutilizzare.

Se un file è aperto in lettura una volta chiuso potrà essere aperto da un altro programma.

Se un file è aperto in scrittura, verrà salvato definitivamente ciò che è stato scritto dall'istruzione fprintf.

Non chiudere un file aperto in scrittura, non salverà ciò che è stato scritto con la fprintf fintantoché il programma non termina.

Funzione feof



La funzione feof() restituisce un valore logico vero nel caso in cui sia stata raggiunta la fine del file e zero in tutti gli altri casi. Il prototipo della feof è il seguente:

```
int feof(FILE *fp);
```

```
while (!feof(fp ))
```

```
.....
```


Riassunto



- Dichiarare una variabile puntatore: **FILE *fp**
- Aprire il file con l'istruzione **fp=fopen("input.txt","rt");**
("wt" o "at")
- Controllare la corretta apertura del file **if(fp==NULL)**
- **while (!feof(fp)) + istruzioni**
- Operazioni di lettura/scrittura
 - **fscanf(fp,"%d", &num);**
 - **fprintf(fp,"%d", num);**
- Chiudere il file **fclose (fp);**

Esercizi di ripasso file di testo



1 – Scrivere un programma che effettui la copia di un file di testo in un altro file trasformando le lettere minuscole in maiuscole e le maiuscole in minuscole e lasciando invariati gli altri caratteri.

2- Generare 50 numeri casuali tra 1 e 50 e scrivere nel file output.txt solo quelli che non si ripetono.



Esercizi ripasso struct

- 1) Creare la struttura “libro” con un codice libro numerico (numero intero), un numero di pagine e un costo. Chiedere i dati di tre libri e calcolare il costo medio per pagina di ogni libro.
 - 2) Modificare l’esercizio precedente in modo da poter inserire n libri.
-

Linguaggio C - I file



Parte II

Accesso al file



Le operazioni di lettura e scrittura su file possono essere effettuate

- accedendo ai dati carattere per carattere
- linea per linea
- blocco per blocco

Di solito si usa l'accesso linea per linea nel caso di file di testo e l'accesso carattere per carattere o blocco per blocco in presenza di file binari.

Tipi di accesso (file testo)



Carattere per carattere

fgetc legge il prossimo carattere del file specificato

fputc scrive come prossimo carattere del file

Accesso per linea

fgets legge linee (stringhe di caratteri terminate da un newline) dal file specificato

fputs scrive linee (stringhe di caratteri terminate da un newline) sul file specificato



	<i>Funzione da console</i>	<i>Funzione da file</i>
<i>a caratteri</i>	<code>int getchar(void);</code> <code>int putchar(int c);</code>	<code>int fgetc(FILE* f);</code> <code>int fputc(int c, FILE* f);</code>
<i>a stringhe di caratteri</i>	<code>char* gets(char* s);</code> <code>int puts(char* s);</code>	<code>char* fgets(char* s, int n, FILE* f);</code> <code>int fputs(char* s, FILE* f);</code>
<i>formattato</i>	<code>int printf(char* fm, expr1,...exprN);</code> <code>int scanf(char* fm, addr1,...addrN);</code>	<code>int fprintf(FILE* f, char*fm, expr1,...exprN);</code> <code>int fscanf(FILE* f, char* fm, addr1,...addrN);</code>



I file binari

Definizione



Un file binario è una **sequenza di byte**, e viene usato per archiviare su memoria di massa qualunque tipo di informazione che non sia un testo.

Si usano file binari per archiviare:

- valori numerici (int, float, double, ...)
- tipi definiti dall'utente (struct, vettori, ...)

Ai file binari si accede per blocco

fread(addr, dim, nElem, fp) legge un blocco di dati binari

fwrite(addr, dim, nElem, fp) scrive in un blocco di dati binari

Libreria standard in C – file binari



Operazione	Funzione file binari
Apertura	fopen
Lettura	fread
Scrittura	fwrite
Chiusura	fclose

Scrittura



fwrite(addr, dim, nElem, fp);

Scrive sul file **nElem** elementi, ognuno pari a **dim** byte (scrive quindi $nElem \times dim$ byte) e restituisce il numero di elementi effettivamente scritti (possono essere meno di **nElem**).

Questa funzione ha quattro argomenti:

- **addr** l'indirizzo del dato da scrivere
- **dim** la sua dimensione
- **nElem** il numero di elementi effettivamente scritti
- **fp** il puntatore al file

Esempio: x è una variabile intera

fwrite(&x, sizeof(x), 1, fp);



fread(addr, dim, nElem, fp);

Legge dal file **nElem** elementi, ognuno pari a **dim** byte (quindi di legge $nElem \times dim$ byte) e restituisce il numero di elementi effettivamente letti (possono essere meno di **nElem**).

Questa funzione ha quattro argomenti:

- **addr** l'indirizzo della variabile su cui scrivere il dato
- **dim** il numero di byte in memoria occupati dal dato
- **nElem** il numero di elementi effettivi
- **fp** il puntatore al file

Esempio: x è una variabile intera

fread(&x, sizeof(x), 1, fp);

Esempio scrittura di due interi



```
#include <stdio.h>

int main()
{
    FILE *fp;
    int x=40, y=50;
    /* apre il file in scrittura */
    fp=fopen("numeri.dat", "wb");
    if( fp==NULL ) {
        printf("Errore in apertura del file");
        return 1;
    }
    /* Scrittura primo numero */
    fwrite(&x, sizeof(int), 1, fp);
    /* Scrittura secondo numero */
    fwrite(&y, sizeof(int), 1, fp);
    /* chiude il file */
    fclose(fp);
    /* stampa i due numeri letti */
    printf("Nel file ci sono %d e %d\n", x, y);

    return 0;
}
```

Esempio lettura 2 numeri



```
int main() {  
    FILE *fp;  
    int x, y;  
    int res;  
    /* apre il file in lettura */  
    fp=fopen("numeri.dat", "rb");  
    if( fp==NULL ) {  
        printf("Errore in apertura del file");  
        return 1;  
    }  
    /* lettura primo numero */  
    fread(&x, sizeof(int), 1, fp);  
    /* lettura secondo numero */  
    fread(&y, sizeof(int), 1, fp);  
    /* chiude il file */  
    fclose(fp);  
    /* stampa i due numeri letti */  
    printf("Numeri letti %d e %d\n", x, y);  
  
    return 0;  
}
```

Leggere un file di testo Posizione.txt contenente righe con le seguenti informazioni:



```
18:49:59,12/12/2010,43°33'00"N,10°19'  
30"E
```

Costruire un file binario Posizione.dat che contenga record con la seguente struttura:

```
struct POSIZIONE  
{  
    int giorno;  
    int mese;  
    int anno;  
    int ora;  
    int minuti;  
    int secondi;  
    int gradi_latitudine;  
    int primi_latitudine;  
    int secondi_latitudine;  
    int gradi_longitudine;  
    int primi_longitudine;  
    int secondi_longitudine;  
};
```

Attenzione:

La latitudine N ha valori positivi mentre S ha valori negativi.

La longitudine E ha valori positivi mentre la longitudine O ha valori negativi.

```
fscanf(sorgente, "%i:%i:%i,", &ora, &minuti, &secondi);
```

Lettura in un ciclo



È possibile leggere da un file binario senza sapere esattamente quanti dati ci sono: si continua a leggere fino al punto in cui si raggiunge la fine del file.

Quando il file è finito la funzione `fread` ritorna 0 (invece che 1).

Si può quindi leggere un file con un ciclo da cui si esce non appena il valore di ritorno di `fread` è zero.

Come nei file di testo solo che invece di `fscanf` si usa `fread`.

Scrivere/leggere un array con una sola istruzione



La funzione **fread** permette di leggere un array da file con una sola istruzione.

Nel caso in cui vogliamo leggere un certo numero di elementi, come terzo parametro alla funzione **fread** si deve usare una variabile intera che conterrà il numero di elementi letti.

Lo stesso vale per **fwrite**



Esempio scrittura interi

Salvare su un file binario il contenuto di un vettore di interi.

```
int main() {  
    FILE *fp;  
    int vet[10] = {1,2,3,4,5,6,7,8,9,10};  
    if ((fp = fopen("numeri.dat", "wb")) == NULL) {  
        printf("Errore di apertura \n");  
        return 1;  
    }  
    fwrite(vet, sizeof(int), 10, fp);  
    fclose(fp);  
    return 0;  
}
```

Esempio lettura interi



Leggere da un file binario degli interi e memorizzarli in un vettore.

```
int main(){  
FILE *fp;  
int vet[40], i, n;  
if ((fp = fopen("numeri.dat", "rb"))==NULL) {  
    printf("Errore di apertura\n");  
    return 1;}  
  
n = fread(vet, sizeof(int), 40, fp);  
for (i=0; i<n; i++) printf("%d ", vet[i]);  
fclose(fp);  
return 0;  
}
```

IMPORTANTE: controllare n, perché si potrebbero leggere meno di 40 numeri.

Esempio di scrittura di una struct



Archiviare in coda ad un file binario una serie di dati relativi a persone (ogni persona è caratterizzata da cognome, nome ed età).

```
typedef struct {
    char cognome[20], nome [20];
    int eta; } persona;

main(){
FILE *fp;
persona p;
int continua;
if ((fp = fopen("archivio.dat","wb"))==NULL) {
    printf("Errore di apertura\n");
    return 1;
}
do {printf("Immettere Cognome, Nome ed età:\t");
    scanf("%s%s%d", p.cognome, p.nome, &p.eta);
    fwrite(&p,sizeof(persona),1,fp);
    printf("continure? (1=si 0=no)");
    scanf("%d",&continua);
}
while (continua);
fclose(fp);
}
```

Esempio di lettura



Leggere da un file binario e mostrare a video i dati relativi alle persone presenti nel file (il tipo `persona` è dato).

```
#include <stdio.h>
#include <stdlib.h>
typedef struct {
    char cognome[20], nome [20];
    int eta; } persona;
int main(){
    FILE *fp;
    persona p;
    if ((fp = fopen("archivio.dat","rb"))==NULL) {
        printf("Errore di apertura\n");
        Return 1;}
    while (fread(&p,sizeof(persona),1,fp)>0)
        printf("Cognome: %s, Nome: %s, Età:%d\n",
            p.cognome, p.nome, p.eta);
    fclose(fp);
    return 0;
}
```



il ciclo termina quando `fread()` restituisce 0



Accesso casuale ai file binari

Accesso casuale al contenuto di file binari



OFFSET= posizione espressa in byte

Poiché i blocchi (record) dei file binari hanno tutti la stessa lunghezza è possibile determinare l'offset di uno specifico blocco (record) per posizionare il puntatore al file su di esso.

Per fare ciò si utilizza la funzione **fseek**:

fseek(fp, offset, da_dove);

offset può essere anche negativa

da_dove può valere SEEK_SET (inizio file), SEEK_CUR (posizione corrente) o SEEK_END (fine file).

In caso di successo, fseek() ritorna 0 altrimenti ritorna -1.

Poi con un operazione di fread o fwrite si legge o scrive il record.

Come calcolare l'offset



Esempio: il file "posizione.dat" memorizza un codice.

```
typedef struct {
    int ora;
    int codice;
} posizione;

int main(){
    FILE *fp;
    posizione p;
    int hour,offset,n;
    if ((fp = fopen("posizione.dat","rb"))==NULL) // CONTROLLO APERTURA FILE CORRETTA
    {
        printf("Errore di apertura\n");
        return 1;
    }
    printf("Immettere ora per sapere il codice"); // LETTURA DELL'ORA DI CUI SI VUOLE CONOSCERE IL CODICE
    scanf("%d",&hour);
    offset=(hour-1)*sizeof(posizione);           // CALCOLO DELL'OFFSET = ORA IN INPUT * DIMENSIONE IN BYTE DELLA STRUTTURA POSIZIONE
    fseek(fp,offset,SEEK_SET);                    // SPOSTAMENTO DEL PUNTATORE AL FILE ALLA POSIZIONE RICHIESTA (+OFFSET DALL'INIZIO DEL FILE)
    fread(&p,sizeof(posizione),1,fp);            // LETTURA DEL BLOCCO (RECORD)
    printf("Codice: %d",p.codice);                // STAMPA DEL CODICE RELATIVO ALL'ORARIO RICHIESTO
    fclose(fp);
    return 0;
}
```

Altre funzioni



La funzione **rewind()** imposta l'indicatore di posizione del file puntato da fp all'inizio file. Non ritorna valori

E' equivalente a:

`fseek(fp, 0, SEEK_SET).`

La funzione **ftell()** ritorna il valore corrente del puntatore al file (fp).

fseek-ftell-rewind



```
int fseek(FILE *fp, long offset, int da_dove);
```

```
long ftell(FILE *fp);
```

```
void rewind(FILE *fp);
```

Esercizi in pre-verifica



1- Leggere in file acquisti.txt che contiene in ogni riga codice, nome, cognome, prezzo e quantità (entrambi interi) in una struttura Acquisti e poi scriverli nel file binario Acquisti.dat. Poi scrivere un programma per leggere e stampare a video il contenuto di Acquisti.dat.

2- Il file binario metrodata.bin ha un record avente la seguente struttura:

```
struct registra{  
    int stazione;  
    int scesi;  
    int saliti;  
}
```

e contiene un record per ogni stazione – identificata da un codice numerico – del percorso di una linea metropolitana. Scrivere un programma che legga il contenuto di metrodata.dat e produca un file di tipo .txt (metrodata.txt) la cui singola riga abbia un formato #101,20+,80- e determinare il numero di persone che sono presenti sulla metro all'ultima stazione.